

## TALLER EVALUATIVO UNIDAD 2: SISTEMAS DISTRIBUIDOS, MIDDLEWARE Y COMUNICACIÓN MULTIMEDIA

# Computación en Internet I

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS  
INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

### INFORMACIÓN DE LA ACTIVIDAD

### IDENTIFICACIÓN

|               |                                  |                 |                                          |
|---------------|----------------------------------|-----------------|------------------------------------------|
| CÓDIGO:       | 09810 - TIC                      | PERÍODO:        | 202620 (Semestre 2026-2)                 |
| NRC / GRUPO:  | 12378 / 12377 (Grupos 001 y 003) | PONDERACIÓN:    | 8.0 % (Taller 2 – Unidad 2)              |
| MODALIDAD:    | Grupos de 3 o 4 Estudiantes      | CALIFICACIÓN:   | _____ / 5.0                              |
| POLÍTICA IAG: | Nivel 3: Colaboración Asistida   | TIPO ACTIVIDAD: | Taller Evaluativo / Proyecto Distribuido |
| ESTUDIANTE 1: | _____                            | CÓDIGO:         | _____                                    |
| ESTUDIANTE 2: | _____                            | CÓDIGO:         | _____                                    |
| ESTUDIANTE 3: | _____                            | CÓDIGO:         | _____                                    |
| ESTUDIANTE 4: | _____                            | CÓDIGO:         | _____                                    |

## 1 Contexto del Problema y Desafío de Ingeniería

En las sesiones previas del curso exploramos la comunicación entre procesos mediante sockets crudos (TCP y UDP). A través de dicha experiencia identificamos las dificultades de construir sistemas a gran escala bajo ese paradigma: la serialización artesanal de bytes, el acoplamiento rígido de mensajes y la complejidad de coordinar múltiples clientes concurrentes. Posteriormente, introdujimos el concepto de **Middleware orientado a objetos y Llamadas a Procedimiento Remoto (RPC)** a través de **ZeroC Ice (Internet Communications Engine)**, comprendiendo cómo los contratos en lenguaje **Slice** permiten definir interfaces fuertemente tipadas y transparentes a la ubicación física de los nodos.

En este taller evaluativo, cada equipo de ingeniería asumirá el desafío de **diseñar y construir una plataforma distribuida de comunicación en tiempo real y colaboración multimedia** (en una arquitectura similar a Discord, Slack o Telegram). El sistema combina estratégicamente dos mundos de comunicación en red:

- Canal de Señalización, Mensajería y Transferencia de Archivos:** Gestionado a través del middleware **ZeroC Ice**, garantizando tipado estricto, despacho asíncrono y fiabilidad sobre TCP.
- Canal de Transmisión de Voz en Tiempo Real:** Gestionado directamente sobre datagramas **UDP**, priorizando la baja latencia y la fluidez acústica sin incurrir en la sobrecarga de retransmisión y confirmación de TCP.

**Nota Pedagógica Crucial:** A diferencia de los talleres introductorios guiados, este documento **no proporciona código de solución ni guías paso a paso**. Cada equipo es autónomo en la formulación de sus contratos Slice (.ice), la arquitectura interna de sus servidores y clientes, el manejo de hilos y la elección de la interfaz de usuario (**CLI interactiva o JavaFX**).

## 2 Requerimientos Funcionales del Sistema

La solución desarrollada por el equipo debe cumplir estrictamente con los siguientes requerimientos funcionales:

### 2.1 RF-01: Gestión de Sesión, Presencia y Registro de Clientes

- **Autenticación e Identificación:** Cada cliente debe conectarse al servidor ZeroC Ice indicando un identificador de usuario (*nickname*) único. Si el identificador ya se encuentra en uso por otra sesión activa, el servidor debe rechazar la conexión arrojando una excepción personalizada de Slice.
- **Registro de Retorno (Callbacks):** Para permitir que el servidor notifique eventos al cliente de forma asíncrona (mensajes entrantes, llamadas, archivos), el cliente debe registrar un proxy de retorno (*callback object*) en el servidor al momento de iniciar sesión.
- **Monitor de Presencia en Tiempo Real:** Todos los clientes conectados deben recibir notificaciones automáticas inmediatas cuando un usuario se conecta o se desconecta de la plataforma.
- **Desconexión Limpia y Tolerancia a Fallos:** Debe implementarse un mecanismo de cierre de sesión ordenado (*logout*) que limpie los recursos y proxies en el servidor, así como detección y remoción de clientes cuya conexión se pierda abruptamente.

### 2.2 RF-02: Mensajería Privada Directa (1 a 1)

- **Comunicación Punto a Punto:** Un usuario conectado debe poder enviar un mensaje de texto privado a cualquier otro usuario disponible en la red indicando su *nickname*.
- **Despacho Asíncrono:** El mensaje debe enviarse al servidor Ice, el cual localizará el proxy de callback del destinatario y despachará el mensaje en tiempo real.
- **Confirmación y Control de Errores:** El remitente debe recibir confirmación de que el mensaje fue entregado al servidor, o un mensaje de error descriptivo en caso de que el destinatario no exista o se haya desconectado.

### 2.3 RF-03: Creación y Gestión de Grupos / Salas de Chat (Canales)

- **Creación Dinámica de Salas:** Cualquier usuario debe tener la capacidad de crear una nueva sala o grupo de chat asignándole un nombre o identificador único.
- **Afiliación y Listado:** Los usuarios pueden consultar el catálogo de salas disponibles, unirse a una o múltiples salas simultáneamente y abandonarlas en cualquier instante.
- **Difusión Grupal (Broadcast):** Los mensajes emitidos dentro de una sala deben propagarse a todos los participantes inscritos activamente en ella, identificando con claridad el remitente y la sala de origen, sin enviarse de vuelta como duplicado al emisor.
- **Aislamiento de Tráfico:** Los mensajes pertenecientes a un grupo jamás deben filtrarse ni entregarse a usuarios que no pertenezcan a dicho grupo.

### 2.4 RF-04: Transferencia de Contenido Multimedia sobre ZeroC Ice

- **Tipos de Contenido Soportados:** Transferencia de archivos binarios tales como imágenes (.png, .jpg), notas de voz (.wav, .mp3) o documentos (.pdf, .txt).
- **Transferencia Vía Contrato Slice:** El envío y recepción de archivos multimedia debe realizarse **estrictamente a través de interfaces y operaciones de ZeroC Ice**, empleando secuencias binarias de Slice (*sequence<byte>*).
- **Estrategia de Fragmentación (Chunking):** Dado que Ice posee un límite de tamaño por mensaje configurado (*Ice.MessageSizeMax*), el sistema debe fragmentar los archivos en bloques o *chunks* sucesivos (ej. paquetes de 32 KB a 64 KB) con metadatos asociados (nombre de archivo, tamaño total, índice de bloque y total de fragmentos).
- **Destino Directo y Grupal:** Los archivos deben poder enviarse tanto a un usuario en chat privado como a una sala grupal completa.
- **Reconstrucción e Integridad:** El cliente receptor debe reconstruir el archivo en disco o carpeta temporal garantizando que no sufra corrupción binaria ni truncamiento.

## 2.5 RF-05: Llamadas de Voz Directas (1 a 1) sobre Datagramas UDP

### ■ Señalización y Negociación Vía Ice:

- El usuario iniciador invoca una operación RPC en el servidor Ice solicitando iniciar una llamada de voz con el usuario destino, publicando su dirección IP y puerto UDP de recepción de audio.
- El usuario destino recibe una notificación de llamada entrante y puede **Aceptar** o **Rechazar** la llamada vía Ice.
- Al aceptar, el destino reporta su IP y puerto UDP asignado, quedando formalmente establecida la sesión de voz.

### ■ Flujo de Audio UDP:

- Captura de audio en tiempo real desde el micrófono del usuario emisor (empleando la API estándar `javax.sound.sampled` de Java).
- Empaquetamiento de muestras de audio en datagramas UDP (`DatagramPacket`) y transmisión continua al endpoint UDP del receptor sin intermediación de retransmisiones TCP.
- Recepción de datagramas UDP en un hilo dedicado en el cliente y reproducción inmediata en los altavoces.

### ■ Finalización de Llamada: Cualquiera de los dos usuarios puede colgar la llamada mediante una señal en Ice, lo cual cerrará de forma segura los sockets UDP y liberará las líneas de audio.

## 2.6 RF-06: Llamadas de Voz Grupales (Conferencias de Audio) sobre UDP

### ■ Inicio y Vinculación a Conferencia: En cualquier sala de chat grupal activa, los usuarios deben poder iniciar o sumarse a una llamada grupal de voz.

### ■ Arquitectura de Distribución de Audio Grupal: El equipo debe diseñar e implementar una de las siguientes dos estrategias (documentando la decisión en el informe):

- Relay Centralizado de Audio (SFU Ligero):** El servidor dispone de un socket UDP que recibe los paquetes de voz de cada emisor y los reenvía a todos los demás participantes de la sala de audio.
- Malla Peer-to-Peer (Mesh):** A través de la señalización de Ice, cada participante obtiene la lista de IPs y puertos UDP de los demás integrantes de la llamada y transmite réplicas de sus datagramas directamente a cada uno.

### ■ Control de Audio: El cliente debe permitir silenciar el micrófono (*Mute / Unmute*) en cualquier momento y abandonar la conferencia sin interrumpir la llamada de los demás participantes.

## 3 Requerimientos No Funcionales y Restricciones de Diseño

### 1. Middleware Obligatorio: Toda la lógica de control, catálogos, señalización, mensajería y transferencia de archivos debe residir sobre **ZeroC Ice (versión 3.7 o superior)** en lenguaje Java.

### 2. Canal de Audio Estrictamente UDP: El streaming de voz debe utilizar exclusivamente sockets de datagramas UDP (`java.net.DatagramSocket`). Queda prohibido transmitir el flujo continuo de audio sobre llamadas síncronas de Ice o conexiones TCP.

### 3. Interfaz de Usuario (A Elección del Equipo):

- **Opción A (CLI Avanzada):** Interfaz por consola con comandos interactivos limpios (ej. `/login`, `/msg <user><txt>`, `/create <room>`, `/join <room>`, `/sendfile <target><path>`, `/call <target>`, `/hangup`, `/mute`). Requiere un hilo de lectura de entrada de usuario desacoplado de la impresión de mensajes asíncronos.
- **Opción B (JavaFX):** Interfaz gráfica de escritorio con paneles para lista de usuarios, salas, ventana de chat, selector de archivos y controles de llamada (botones de llamar, contestar, colgar y mute).
- **Nota de Evaluación:** Ambas opciones se evalúan bajo el mismo estándar de calidad. Una CLI robusta, bien formateada y sin bloqueos de concurrencia tiene exactamente la misma ponderación que una interfaz JavaFX.

4. **Concurrencia Segura en el Servidor (Thread-Safety):** El servidor Ice es intrínsecamente multihilo; las colecciones que almacenan el estado global (mapa de usuarios conectados, proxies de clientes, salas activas, participantes en llamadas) deben estar protegidas rigurosamente contra condiciones de carrera mediante sincronización explícita o colecciones concurrentes (`ConcurrentHashMap`, `CopyOnWriteArrayList`).
5. **Gestión del Proyecto con Gradle:** La solución debe organizarse como un proyecto multimodular en **Gradle** estructurado con el plugin oficial de Ice (`com.zeroc.gradle.ice-builder.slice`) y scripts de ejecución definidos en el archivo `build.gradle` para servidor y cliente.

#### 4 Cuestionario de Análisis Conceptual y Decisiones de Diseño

Como parte del informe técnico de entrega, el equipo debe responder con profundidad técnica y justificación de ingeniería a las siguientes interrogantes:

1. **Arquitectura Híbrida TCP/UDP:** ¿Cuáles son las razones técnicas fundamentales para diseñar una arquitectura híbrida donde el control, el chat de texto y la transferencia de archivos corren sobre ZeroC Ice (TCP), mientras que el flujo de voz corre directamente sobre UDP? ¿Qué consecuencias acústicas y de rendimiento ocurrirían si la voz se transmitiera sobre Ice/TCP bajo condiciones de fluctuación de red (*jitter*) o pérdida de paquetes?
2. **Manejo de Buffer y Chunking en Ice:** Si un usuario intenta transferir un video de 80 MB mediante una única llamada RPC pasando toda la secuencia de bytes en un parámetro `sequence<byte>`, ¿qué excepción o comportamiento manifestará ZeroC Ice por defecto? Explique cómo la técnica de fragmentación en bloques resuelve este problema y qué parámetros de configuración de Ice intervienen.
3. **Escalabilidad en Llamadas Grupales (Relay vs. Mesh):** Analice comparativamente la topología de Audio Relay centralizado frente a la topología Mesh Peer-to-Peer elegida por su equipo. Si una sala cuenta con  $N$  participantes activos hablando simultáneamente, ¿cuál es el número total de paquetes UDP que debe emitir cada cliente y cuál es la carga total que soporta la red del servidor en ambas alternativas?
4. **Transparencia y Callbacks en Ice:** Describa el ciclo de vida y la configuración de adaptadores de objetos requerida en el cliente para permitir que el servidor Ice invoque métodos de retorno (*callbacks*) sobre él. ¿Qué papel juegan los proxies directos vs indirectos en este flujo?

#### 5 Rúbrica de Evaluación Modularizada por Entregable (Escala 0.0 a 5.0)

La evaluación del taller se realizará de forma modular con base en la demostración funcional en vivo, el código fuente y las respuestas técnicas del equipo durante la sustentación:

| Criterio / Módulo                                       | Pts        | Excelente (4.5 – 5.0)                                                                                                                                                                                                                            | Aceptable (3.0 – 4.4)                                                                                                                                                                  | Insuficiente (0.0 – 2.9)                                                                                                                        |
|---------------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Contratos Slice y Arquitectura Distribuida</b>    | <b>0.7</b> | Diseño formal y modular del archivo .ice. Define limpiamente interfaces para servidor y callbacks, structs estructurados y excepciones personalizadas. Proyecto Gradle multimódulo limpio con tareas de ejecución independientes.                | Contrato Slice funcional pero con definiciones monolíticas, tipos genéricos o escaso manejo de excepciones. Proyecto Gradle con dependencias rígidas o errores menores de compilación. | Contrato Slice inexistente, incompleto o con errores graves de sintaxis que impiden compilar los stubs en Java. Arquitectura no modular.        |
| <b>2. Sesión, Presencia y Chat Directo (1 a 1)</b>      | <b>0.8</b> | Registro con nickname único, detección de duplicados y monitor de presencia en tiempo real reactivo ante conexiones/desconexiones. Mensajería privada 1 a 1 fluida, asíncrona y con confirmaciones claras sin bloqueos.                          | Inicio de sesión y chat directo funcionan, pero la lista de presencia presenta retrasos, no notifica desconexiones o requiere refresco manual.                                         | Falla el registro de usuarios, se permiten nombres duplicados o la mensajería privada 1 a 1 no entrega los mensajes al destinatario.            |
| <b>3. Salas Grupales y Difusión (Broadcast)</b>         | <b>0.8</b> | Creación, listado y vinculación dinámica a salas de chat. Propagación instantánea de mensajes a todos los miembros de la sala sin duplicados al emisor y con estricto aislamiento entre grupos distintos.                                        | Las salas de chat funcionan pero presentan fallas al salir usuarios, mensajes duplicados esporádicos o retrasos en la difusión concurrente.                                            | Los mensajes se mezclan entre diferentes salas de chat, el broadcast falla ante múltiples usuarios o la creación de grupos colapsa el servidor. |
| <b>4. Transferencia Multimedia sobre Ice (Chunking)</b> | <b>0.9</b> | Envío de imágenes/archivos binarios a través de Ice utilizando secuencias de bytes y estrategia de fragmentación ( <i>chunking</i> ). Reconstrucción íntegra y almacenamiento en el receptor tanto en chat directo como en salas.                | Transfiere archivos pequeños correctamente, pero falla con archivos de mediano tamaño por desborde de buffer ( <i>MemoryLimitException</i> ) o faltan metadatos.                       | No implementa transferencia de archivos sobre Ice, utiliza sockets TCP alternos no autorizados o los archivos recibidos resultan corruptos.     |
| <b>5. Llamadas de Voz UDP (Directas y Grupales)</b>     | <b>1.2</b> | Señalización completa en Ice (solicitar, timbrar, aceptar, rechazar y finalizar llamada). Transmisión continua de voz sobre datagramas UDP con baja latencia y audio inteligible. Soporta llamada 1 a 1 y conferencia grupal con mute funcional. | Llamada directa 1 a 1 operativa sobre UDP, pero la llamada grupal presenta eco severo, bloqueos al ingresar más de 2 participantes o fallas en la finalización.                        | No realiza señalización de llamadas en Ice, no hay transmisión funcional de audio sobre UDP o utiliza protocolos síncronos inadecuados.         |
| <b>6. Concurrencia, Calidad de UI y Sustentación</b>    | <b>0.6</b> | Servidor plenamente thread-safe sin condiciones de carrera. Interfaz (CLI o JavaFX) altamente reactiva, intuitiva y profesional. README detallado. Sustentación oral sólida con dominio técnico por todos los integrantes.                       | Servidor funcional pero con bloqueos ocasionales por sincronización; UI básica con formato desordenado. Dominio técnico desigual entre los miembros del equipo.                        | Servidor colapsa por concurrencia no controlada. Interfaz inusable. Documentación deficiente y desconocimiento del código en la sustentación.   |
| <b>CALIFICACIÓN TOTAL</b>                               | <b>5.0</b> | <b>Escala Institucional Universidad Icesi: Aprobatoria <math>\geq 3.0</math> • NRC: 12378 / 12377</b>                                                                                                                                            |                                                                                                                                                                                        |                                                                                                                                                 |

## INSTRUCCIONES DE ENTREGA Y METODOLOGÍA

MODALIDAD: EQUIPOS DE 3 O 4

- Conformación de Equipos:** La actividad se desarrollará en grupos conformados estrictamente por **3 o 4 estudiantes**. Cada integrante debe participar activamente en el diseño arquitectónico y la codificación.
- Repositorio de Código (Git):** El código fuente completo debe alojarse en un repositorio privado (GitHub o GitLab) con permisos de lectura para el profesor. Debe incluir el historial claro de commits de todos los integrantes.
- Estructura del Proyecto:** El repositorio debe contener el archivo `build.gradle` y los scripts wrapper (`gradlew` y `gradlew.bat`) para que cualquier evaluador pueda clonar y ejecutar el servidor y los clientes mediante:

```
./gradlew runServer
./gradlew runClient
```

- Documentación (README.md):** El repositorio debe incluir un manual técnico en formato Markdown con:

- Diagrama de arquitectura del sistema indicando el flujo de Ice (señalización/mensajes/archivos) y el flujo UDP (voz).
- Manual de usuario detallando los comandos (si es CLI) o la guía de uso (si es JavaFX).

- Respuestas formales al Cuestionario de Análisis Conceptual (Sección 4 de este documento).
5. **Bitácora de Uso Ético de IAG (Nivel 3 - Colaboración Asistida):** De acuerdo con el programa oficial del curso, se permite el uso asistido de Inteligencia Artificial Generativa. Es obligatorio incluir en el informe una sección de bitácora que resuma los prompts principales empleados, el código generado que fue adaptado críticamente y las lecciones aprendidas durante la depuración.
  6. **Sustentación y Demostración Práctica:** Cada equipo realizará una sustentación en vivo frente al docente en el laboratorio, conectando múltiples clientes simultáneos desde diferentes máquinas de la red local para verificar la concurrencia, el intercambio multimedia y la calidad acústica de las llamadas UDP.